A PRACTICAL 2026 FIELD GUIDE

Claude Code & OpenAI Codex

From your very first command to confident, everyday use — the two leading agentic coding tools explained in plain language, with real examples you can run today.

TOOL ONE

Claude Code by Anthropic

TOOL TWO

Codex by OpenAI

For complete beginners & curious developers

Updated May 2026

Contents

Introduction — What is an "agentic" coding tool?

If you have never used a tool like this before, start here. Five minutes of background will make everything that follows click into place.

For years, AI helped with code by *suggesting the next line* as you typed — autocomplete on steroids. Claude Code and OpenAI Codex are a different species. They are **agentic**, which means you describe a goal in plain English and the tool plans the work, reads the relevant files, writes or edits code across many files, runs commands and tests, sees the results, and corrects itself — all in a loop, with you approving the important steps.

A helpful mental model: a traditional assistant is like a dictionary you flip through; an agentic coding tool is like a junior engineer sitting next to you. You don't hand it individual words — you hand it a task ("add a login page", "find why the build is failing", "write tests for the payment module") and it goes and does it, showing you its work.

The two tools at a glance

	CLAUDE CODE	OPENAI CODEX
Maker	Anthropic	OpenAI
Engine	Claude models (Opus / Sonnet / Haiku)	GPT-5 series (GPT-5.5, GPT-5.4, GPT-5.3-Codex)
Lives in	Terminal, VS Code, JetBrains, desktop app, web, Slack	Terminal (CLI), VS Code/Cursor/Windsurf, desktop app, cloud, ChatGPT
Access	Claude Pro / Max / Team / Enterprise, or API credits	ChatGPT Plus / Pro / Business / Edu / Enterprise, or API key
Project memory file	`CLAUDE.md`	`AGENTS.md`
Open source CLI?	No (closed binary)	Yes (Rust, on GitHub)

HOW TO READ THIS GUIDE

You do *not* need to pick one before reading. Part 1 covers Claude Code completely, Part 2 covers Codex completely, and they mirror each other section-for-section so you can compare. If you only care about one tool, read its part plus Parts 3–5. Every command shown is something you can type yourself.

ONE SAFETY IDEA TO CARRY THROUGHOUT

Both tools can edit files and run real commands on your computer. That power is the point — but always work inside a **Git repository** so every change is tracked and reversible, and read the permission/approval sections (1.9 and 2.5) before letting either tool run unattended. A good habit: commit your work before handing a big task to the agent.

PART ONE

Claude Code

Anthropic's agentic coding tool — it reads your codebase, edits files, runs commands, and works wherever you do.

1.1 What Claude Code is

Claude Code is an AI coding assistant that understands your *entire* project, not just the file you have open. You talk to it in natural language and it takes real action: planning a feature, writing code across multiple files, running your test suite, fixing what fails, and even creating Git commits and pull requests.

It started life in the terminal, but in 2026 it is one engine behind several "surfaces": the command-line tool (the original and most powerful), a VS Code / JetBrains extension, a standalone desktop app, a web version at `claude.ai/code`, and integrations like Slack and GitHub Actions. Crucially, **all surfaces share the same settings, the same `CLAUDE.md` files, and the same connected tools**, so what you learn in one place carries everywhere.

WHAT IT'S ESPECIALLY GOOD AT

- **Building features from a description** — it plans, writes the code, and checks that it works.
- **Debugging** — paste an error or describe a symptom; it traces the cause through your codebase and fixes it.
- **Navigating unfamiliar code** — ask "where does login happen?" and get a real answer with file references.
- **Tedious chores** — fixing lint errors, resolving merge conflicts, updating dependencies, writing release notes.
- **Git workflows** — staging, committing, branching, and opening pull requests, all conversationally.

1.2 Installation on every platform

You need two things: a terminal, and a Claude account (a **Pro, Max, Team, or Enterprise** subscription is recommended, or an Anthropic Console account with API credits). Node.js 18+ is the only system prerequisite for some setups, but the recommended installers below bundle everything.

> **NOTE**
>
> Installing through `npm` is now *deprecated*. Use the native installer, Homebrew, or WinGet below — these auto-update (native) or are easy to upgrade.

MACOS, LINUX, OR WSL — NATIVE INSTALLER (RECOMMENDED)

```
# One line. Native installs auto-update in the background.
curl -fsSL https://claude.ai/install.sh | bash
```

MACOS — HOMEBREW

```
brew install --cask claude-code        # stable channel
brew install --cask claude-code@latest # newest releases
# Homebrew does not auto-update Claude Code:
brew upgrade claude-code
```

WINDOWS — POWERSHELL

```
irm https://claude.ai/install.ps1 | iex
```

WINDOWS — COMMAND PROMPT (CMD)

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd
```

WINDOWS — WINGET

```
winget install Anthropic.ClaudeCode
# Upgrade later with:  winget upgrade Anthropic.ClaudeCode
```

> **WINDOWS TIP**
>
> Install **Git for Windows** so Claude Code can use the Bash tool; otherwise it falls back to PowerShell. If a command errors with `'&&' is not a valid statement separator` you are in PowerShell, not CMD — your prompt shows `PS C:\` in PowerShell.

Linux users can also install via `apt`, `dnf`, or `apk` on Debian, Fedora/RHEL, and Alpine. To update any install manually, run `claude update`.

1.3 Logging in & your first session

Move into a project folder and launch:

```
cd /path/to/your/project
claude
```

On first run you'll be prompted to log in; complete it in your browser. Your credentials are then stored, so you won't log in again. To switch accounts later, type `/login` inside a session, or from the shell use `claude auth login` (add `--console` to bill API usage instead of a subscription).

You'll see a welcome screen. Now just **talk to it**. Try asking about the code before changing anything:

```
> what does this project do?
> what technologies does this project use?
> where is the main entry point?
> explain the folder structure
```

Then make your first edit. Claude will find the file, *show you the proposed change*, and ask permission before writing:

```
> add a hello world function to the main file
```

> **BEGINNER TIP**
>
> Type `/` at any time to see all commands. Press `↑` for history, `Tab` to autocomplete, and `Shift`+`Tab` to cycle permission modes. Type `/help` if you're ever lost.

1.4 How it works: the agentic loop

Understanding this loop is the single most useful thing for a beginner, because it explains *why* Claude Code behaves the way it does.

1. **You give a goal** in plain language.
2. **Claude gathers context** — it reads relevant files, searches your codebase, and may look things up on the web. You don't paste files in manually; it fetches what it needs.
3. **It plans and acts** — proposes edits, runs commands, and uses tools. Each action that changes something asks for your approval (unless you've relaxed permissions).
4. **It observes the result** — reads command output and test results.
5. **It iterates** — if a test failed, it fixes and tries again, repeating until the goal is met or it needs your input.

The tools it uses behind the scenes include reading and editing files, running shell commands (Bash), searching, and fetching web pages. Connected tools (via MCP, see 1.8) extend this to things like Google Drive, Jira, and Figma.

> **WHY IT SOMETIMES PAUSES**
>
> When Claude asks "Allow this edit?" or "Run this command?", that's the loop hitting a step that changes your machine. This is a feature: you stay in control. Section 1.9 shows how to tune how often it asks.

1.5 Essential commands & flags

There are two kinds of commands: **shell commands** you type in your terminal (they start with `claude`), and **slash commands** you type inside a running session (they start with `/`).

Shell commands

COMMAND	WHAT IT DOES
`claude`	Start an interactive session
`claude "task"`	Start a session with an initial prompt
`claude -p "query"`	Run a one-off query and exit (great for scripts)
`claude -c`	Continue the most recent conversation here
`claude -r "name" "task"`	Resume a session by name or ID
`cat file | claude -p "…"`	Pipe content in for analysis
`claude update`	Update to the latest version
`claude mcp`	Manage connected MCP tools

Useful flags

FLAG	EFFECT
`--model opus`	Pick the model (alias or full name) for this session
`--permission-mode plan`	Start in a specific permission mode (see 1.9)
`--add-dir ../lib`	Give Claude access to extra folders
`--effort high`	Tell the model to think harder (see 1.10)
`-w feature-x`	Work in an isolated Git worktree
`--verbose`	Show full turn-by-turn output
`--dangerously-skip-permissions`	Skip approval prompts — use only in sandboxes

Common slash commands (inside a session)

SLASH COMMAND	WHAT IT DOES
`/help`	List everything available
`/clear`	Clear the conversation and free up context
`/model`	Switch models mid-session

`/resume`	Pick a previous conversation to continue
`/login`	Switch or re-authenticate your account
`/schedule`	Set up a recurring task (a "routine")
`/desktop`	Hand the session to the desktop app for visual diffs

1.6 Everyday workflows, step by step

Here are the tasks you'll do most, with the exact words to type. Talk to Claude like a capable colleague: describe the *outcome*, not the keystrokes.

A. Understand a codebase you've never seen

```
> give me a high-level tour of this repo: the main modules,
  how they fit together, and where requests enter the system
```

Follow up with targeted questions: *"how is authentication handled?"* or *"draw the data flow for placing an order."*

B. Fix a bug

Be specific — a vague prompt gets a vague fix.

```
# Weak:  fix the login bug
# Strong:
> users see a blank screen after entering wrong credentials.
  Find the root cause and fix it, then add a test that covers it.
```

You can also paste an error message directly; Claude will trace it.

C. Build a feature in steps

Breaking a big task into numbered steps produces far better results:

```
> 1. create a database table for user profiles
  2. add API endpoints to get and update a profile
  3. build a page where users can view and edit their info
```

D. Write tests & refactor

```
> write unit tests for the calculator functions, run them,
  and fix any failures
> refactor the auth module to use async/await instead of callbacks
```

E. Git, conversationally

```
> what files have I changed?
> create a branch called feature/profiles
> commit my changes with a descriptive message
> help me resolve these merge conflicts
> open a pull request summarizing what changed and why
```

F. Pipe & automate (the Unix way)

Because `claude -p` reads stdin and prints to stdout, it composes with other tools and fits into CI pipelines:

```
# Watch a log and get alerted
tail -200 app.log | claude -p "summarize any anomalies you see"

# Review only the files changed on this branch
git diff main --name-only | claude -p "review these for security issues"
```

> **THE GOLDEN RULE OF PROMPTING**
>
> Specificity beats length. "Fix the bug where the cart total ignores discounts" works; "fix the cart" does not. When in doubt, tell Claude the symptom, the expected behaviour, and to add a test.

1.7 CLAUDE.md & project memory

`CLAUDE.md` is a plain Markdown file you put in your project root. Claude Code reads it at the start of *every* session, so it's where you record the things you'd otherwise repeat constantly: coding standards, the architecture in one paragraph, preferred libraries, how to run the build and tests, and review rules.

```
# CLAUDE.md — example
## Project
A Next.js + TypeScript storefront. API in /server, UI in /app.

## Conventions
- Use functional React components and hooks only.
- Prefer `pnpm` over npm.
- All money values are integers (cents), never floats.

## Commands
- Build:  pnpm build
- Test:   pnpm test
- Lint:   pnpm lint

## Review checklist
- No secrets in code. Validate all user input. Add tests for new logic.
```

> **A HABIT WORTH FORMING**
>
> Whenever Claude does something the wrong way and you correct it, add a one-line rule to `CLAUDE.md`. Over time the file becomes your project's institutional memory and mistakes stop repeating. Keep it short — roughly 100 lines — because it loads every session and uses context budget.

Claude Code also builds **auto memory** as it works, saving learnings (like the right build command or a tricky debugging insight) across sessions without you writing anything. Configuration lives in a hidden `.claude` directory in your project, and a global one in your home folder.

1.8 Skills, hooks, subagents, MCP & plugins

These five extension points turn Claude Code from a helper into a tailored teammate. You don't need them on day one, but knowing they exist helps you grow into the tool.

Skills & custom commands

A **skill** packages a repeatable workflow your team can share — for example a `/review-pr` or `/deploy-staging` command. You invoke them like any slash command. They're ideal for encoding "the way we do X here."

Hooks

Hooks run your own shell commands automatically around Claude's actions — for instance, auto-formatting after every file edit, or running lint before a commit. They're how you enforce project rules without trusting the model to remember them.

```
# Conceptual: a hook that formats code after each edit
# (configured in settings; runs `prettier` on changed files)
```

Subagents & agent teams

For big tasks, Claude can spawn **subagents** that work on different parts in parallel, with a lead agent coordinating and merging results. **Background agents** let several full sessions run at once while you watch from one screen (`claude agents`).

MCP — connecting external tools

The **Model Context Protocol** is an open standard for plugging AI tools into outside data sources. With MCP, Claude can read design docs in Google Drive, update Jira tickets, pull from Slack, or call your own internal tooling.

```
# Add and inspect MCP servers
claude mcp
```

Plugins

Plugins bundle skills, subagents, hooks, and MCP servers into one installable package — perfect for sharing a complete integration (say, with GitHub or your company's tools) across a team.

WHEN TO REACH FOR WHICH

Use **CLAUDE.md** for facts Claude should always know; **skills** for repeatable commands; **hooks** for automatic local enforcement; **MCP** to connect outside systems; **plugins** to package and share all of the above.

1.9 Permission modes & staying safe

Claude Code asks before doing anything that changes your machine. You control how often it asks by choosing a **permission mode** — cycle them live with `Shift`+`Tab`, or start in one with `--permission-mode`.

MODE	BEHAVIOUR
`default`	Asks before edits and commands (the safe starting point)
`plan`	Read-only: Claude investigates and proposes a plan but changes nothing
`acceptEdits`	Auto-accepts file edits but still asks for commands
`auto`	A background classifier handles routine approvals for you
`bypassPermissions`	Skips all checks — only in throwaway/sandboxed environments

REAL RISK, REAL PRECAUTION

An autonomous agent with shell access can do real damage if pointed at the wrong thing — there have been documented cases of data loss when agents ran freely against production credentials. Protect yourself: work on a **separate Git branch**, make a checkpoint commit before big tasks, never run in `bypassPermissions` on production machines, and prefer containers, VMs, or read-only branches for unattended runs. Claude Code also offers a sandboxed Bash tool that isolates the filesystem and network for safer autonomy.

1.10 Models, effort & cost

Claude Code runs on Claude models. You usually refer to them by **alias** rather than full name, and can switch any time with `/model` or `--model`.

ALIAS	USE IT FOR
`opus`	The most capable model — hard reasoning, large refactors, tricky bugs
`sonnet`	Fast and strong for everyday coding; the common default on Pro
`opusplan`	Plans with Opus, then executes with a faster model — a cost/quality blend

As of mid-2026 the current generation includes Claude Opus 4.8 (newest), Opus 4.7, Opus 4.6, Sonnet 4.6, and Haiku 4.5; you can also pin a full name like `--model claude-sonnet-4-6`. Models change often, so when in doubt run `/model` to see what's available to your account.

Two more dials: the **effort level** (`--effort low|medium|high|xhigh|max`) trades speed for deeper reasoning, and a **fast mode** prioritises quick responses. To watch spending, use the cost-tracking features and consider clearing context with `/clear` between unrelated tasks, since a smaller context is cheaper and sharper.

1.11 Beyond the terminal

The CLI is the heart, but the same engine runs in many places. Pick what fits the moment:

I WANT TO…	USE
Stay in my editor with inline diffs	VS Code or JetBrains extension
Review changes visually, run sessions side by side	Desktop app (or hand off with `/desktop`)
Kick off long tasks with no local setup	Web at `claude.ai/code`
Continue a session from my phone	Remote Control / Claude mobile app
Auto-review every pull request	GitHub Actions or GitLab CI/CD
Turn a Slack bug report into a PR	Mention `@Claude` in Slack
Run work on a schedule	Routines (`/schedule`) or desktop scheduled tasks
Debug a live web app	Claude Code with Chrome

A nice trick: start a long job on the web or phone, then pull it into your terminal later with `claude --teleport`. Sessions follow you across devices.

PART TWO

OpenAI Codex

OpenAI's agentic coding agent — open source, built in Rust, and available in your terminal, IDE, desktop app, the cloud, and ChatGPT.

2.1 What Codex is

Codex is OpenAI's coding agent. Like Claude Code, you give it a goal in plain language and it reads, changes, and runs code on your machine inside a folder you choose. The terminal version (**Codex CLI**) is **open source and written in Rust** for speed, and it's joined by an IDE extension, a desktop app, a cloud version, and a presence inside ChatGPT.

Codex is included with **ChatGPT Plus, Pro, Business, Edu, and Enterprise** plans; you can also use it with API credits by signing in with an OpenAI API key. The same instruction files and tool connections work across all of its surfaces.

WHAT IT'S ESPECIALLY GOOD AT

- **Implementation, refactors, debugging, testing** — its core strengths.
- **Reading screenshots & design specs** — attach an image and it works from it.
- **Long-horizon tasks** — multi-step jobs that run for a while, including overnight in the cloud.
- **Reviewing its own work** — a separate review agent can check changes before you commit.
- **Computer use** — on supported setups it can operate desktop apps by seeing, clicking, and typing.

2.2 Installation: CLI, IDE, app & cloud

Codex has four ways in. Most beginners start with either the **app** (friendliest) or the **CLI** (most scriptable).

CLI (terminal)

```
# Install globally with npm…
npm i -g @openai/codex

# …or with Homebrew on macOS
brew install codex

# Run it (first run prompts you to sign in)
codex

# Upgrade later
npm i -g @openai/codex@latest
```

> **PLATFORM NOTE**
>
> The Codex CLI runs on macOS and Linux. Windows support is *experimental* — for the best experience run Codex inside a **WSL** workspace and follow OpenAI's Windows setup guide.

IDE extension (VS Code, Cursor, Windsurf)

Install the Codex extension from your editor's marketplace (search "Codex" or use the install links in the docs). It appears in the sidebar; sign in with your ChatGPT account or an API key. It starts in **Agent mode** by default, so it can read files, run commands, and write changes in your project.

Desktop app & cloud

Download the Codex app (macOS, Windows, with Linux on the way), open it, sign in, and choose a project folder. Make sure **Local** is selected to work on your machine. The **cloud** version runs tasks on OpenAI infrastructure in your browser — useful for long jobs and for repos you don't have checked out locally.

2.3 Logging in & your first session

Whichever surface you use, the first launch asks you to authenticate with your **ChatGPT account** (recommended) or an **OpenAI API key**. Note that signing in with an API key disables a few features such as cloud threads.

In the CLI, just run `codex` in your project and start typing. Good first prompts:

```
> Tell me about this project.
> Build a classic Snake game in this repo.
> Find and fix bugs in my codebase with minimal,
  high-confidence changes.
```

> **MAKE A CHECKPOINT FIRST**
>
> Because Codex can modify your codebase, create a Git checkpoint before and after each task: `git add -A && git commit -m "before codex"`. If you don't like the result, you can revert cleanly.

2.4 Models & reasoning levels

Codex runs on OpenAI's GPT-5 series. Switch models mid-session with `/model`, or set one at launch with `codex --model …`. You can also dial **reasoning effort** up or down — higher effort means deeper thinking and slower, more thorough work.

MODEL	NOTES
`gpt-5.5`	Newest frontier model (April 2026); recommended for most Codex tasks — strong and notably token-efficient
`gpt-5.4`	Previous default; reliable general workhorse
`gpt-5.3-codex`	Coding-specialised variant tuned for speed and repository work

```
# Pick a model at launch
codex --model gpt-5.5

# Or switch inside a session
> /model
```

If you don't see the newest model yet, update the CLI, extension, or app — rollouts are gradual.

2.5 Approval modes & sandboxing

Codex's safety model centres on **approval modes** paired with a **sandbox**. The approval mode decides when Codex pauses to ask you; the sandbox limits what it can touch even when it acts.

APPROVAL MODE	BEHAVIOUR
Read Only	Codex can read and plan but won't edit files or run commands without asking
Auto (default)	Edits files and runs commands inside the working folder automatically, but asks before anything outside it or anything network-touching
Full Access	Acts without prompts, including outside the workspace — reserve for sandboxes you trust

You choose the mode when you start, and can change it in the session. From the CLI you'll also see flags for non-interactive runs:

```
# Let it work fully autonomously (sandboxed) — use with care
codex --full-auto "add unit tests for the utils module"

# Control the sandbox explicitly
codex --sandbox workspace-write "refactor the parser"
```

SAME RULE AS CLAUDE CODE

Autonomy is powerful but unforgiving. Keep work in Git, prefer the default **Auto** mode while learning, and only grant **Full Access** in disposable environments. Read OpenAI's sandboxing and "agent approvals & security" pages before unattended runs.

2.6 Everyday workflows, step by step

The prompting style mirrors Claude Code: describe the outcome and constraints clearly.

A. Explore & understand

```
> Summarize this repo's architecture and list the entry points.
> Where is user authentication implemented, and how are
  sessions stored?
```

B. Fix bugs with minimal changes

```
> The /checkout endpoint returns 500 when the cart is empty.
  Reproduce it, find the root cause, and fix it with the
  smallest safe change. Add a regression test.
```

C. Build something new

```
> Build a REST endpoint POST /api/profiles that validates
  input, writes to the profiles table, and returns the new
  record. Include tests and update the OpenAPI spec.
```

D. Work from an image

Attach a screenshot or Figma export and ask Codex to match it:

```
> [attach mockup.png] Build this settings page as a React
  component using our existing design tokens.
```

E. Review before you ship

Have a *separate* Codex agent review your changes — a second pair of eyes that catches what the author missed:

```
> /review        # reviews your current diff in the CLI
```

F. Web search for current info

```
> Check the latest stable version of the express package
  and update package.json, then run the tests.
```

2.7 AGENTS.md & config.toml

Codex reads project instructions from **`AGENTS.md`** — an open, tool-neutral standard (the same file other agents can read too). Place it in your repo root, and add nested ones in subfolders for area-specific rules. It plays the same role as Claude's `CLAUDE.md`.

```
# AGENTS.md — example
# Project
Python FastAPI service. Source in app/, tests in tests/.

# How to work
- Run tests with `pytest -q` before declaring done.
- Use type hints everywhere; run `ruff` to lint.
- Never edit files under app/generated/ — they are auto-built.

# Style
- Small, focused commits with clear messages.
```

Global behaviour lives in **`~/.codex/config.toml`** — model defaults, approval settings, MCP servers, and more.

```
# ~/.codex/config.toml — example
model = "gpt-5.5"
approval_policy = "auto"

[sandbox]
mode = "workspace-write"
```

> **MIGRATING FROM ANOTHER TOOL?**
>
> Codex can import supported instruction files, MCP configuration, skills, and subagents. If you already maintain a `CLAUDE.md`, much of it can become your `AGENTS.md` with light edits.

2.8 Skills, MCP, subagents & hooks

Codex offers the same family of extension points as Claude Code, so concepts transfer directly.

Skills

Skills package reusable instructions and helper scripts Codex can invoke for recurring jobs — for example a documented routine for cutting a release or modernising legacy code.

MCP — connecting tools

Codex speaks the **Model Context Protocol**, giving it access to third-party tools and data sources. Configure servers in `config.toml` or via the MCP commands; this is how Codex reaches systems like issue trackers and docs.

Subagents

Subagents let Codex parallelise complex tasks, delegating independent pieces and combining the results — handy for large refactors or multi-package changes.

Hooks & plugins

Hooks run commands around Codex's actions (format, lint, notify), and **plugins** bundle skills, hooks, and MCP servers for sharing across a team.

2.9 Scripting with codex exec & the SDK

For automation, `codex exec` runs Codex **non-interactively** — give it a prompt, it does the work and exits, printing results. This is the building block for CI jobs and scripts.

```
# Run a task headlessly and capture the result
codex exec "update the changelog from the last 10 commits"

# Pipe context in, like any Unix tool
git log --oneline -20 | codex exec "draft release notes"
```

For deeper integration, the **Codex SDK** (and the app-server) let you embed Codex-powered agents in your own programs, and a **GitHub Action** wires Codex into pull-request workflows.

2.10 Cloud, GitHub & the IDE extension

Three surfaces are worth knowing as you scale up:

- **Codex Cloud** — launch tasks that run on OpenAI infrastructure, even from the CLI; when done, review the diff and apply it, or open a pull request. Great for long or parallel jobs and for keeping your laptop free.
- **GitHub integration** — connect a repo so Codex can review PRs, pick up issues, and push branches. After a cloud task you can `git fetch` and check out the branch to test locally.
- **IDE extension** — keeps you in your editor with a diff view; iterate on results and open a PR without leaving VS Code, Cursor, or Windsurf.

It also integrates with **Slack** and **Linear**, so tasks can start from where your team already works.

PART THREE

Side-by-side

The same job, two tools. How Claude Code and OpenAI Codex line up — and how to choose.

3.1 Feature comparison

TOPIC	CLAUDE CODE	OPENAI CODEX
Maker / engine	Anthropic · Claude (Opus/Sonnet/Haiku)	OpenAI · GPT-5 series (5.5 / 5.4 / 5.3-Co-dex)
Install (terminal)	`curl … install.sh | bash`, Homebrew, WinGet	`npm i -g @openai/codex`, Homebrew
Open-source CLI	No	Yes (Rust)
One-off / headless	`claude -p "…"`	`codex exec "…"`
Project instructions	`CLAUDE.md` + auto memory	`AGENTS.md` (open standard)
Global config	`.claude/` + `settings.json`	`~/.codex/config.toml`
Switch model	`/model` · `--model opus`	`/model` · `--model gpt-5.5`
Safety control	Permission modes (`Shift`+`Tab`)	Approval modes + sandbox
Read-only planning	`plan` mode	**Read Only** mode
Extensibility	Skills, hooks, subagents, MCP, plugins	Skills, hooks, subagents, MCP, plugins
Image input	Yes	Yes
Cloud / async	Web at `claude.ai/code`, routines	Codex Cloud, automations

Editors	VS Code, JetBrains	VS Code, Cursor, Windsurf
Chat suite tie-in	Claude apps, Slack	ChatGPT, Slack, Linear
Access	Claude Pro/Max/Team/Enterprise or API	ChatGPT Plus/Pro/Business/Edu/Enterprise or API

3.2 How to choose

Honestly, the two are close enough that the deciding factor is usually which ecosystem you already pay for and which model you prefer. A few practical pointers:

- **You already have Claude Pro/Max** → start with Claude Code. Already on ChatGPT Plus/Pro → start with Codex. There's no need to buy a second subscription to try the category.
- **You want an open-source, hackable CLI** → Codex is open source.
- **You live across many surfaces (phone, web, Slack, CI)** → both are strong; Claude Code's "follow you across devices" and Codex's cloud tasks are both excellent.
- **The real win** is learning the *workflow* — clear prompts, Git checkpoints, project instruction files, and sensible permissions. Those skills transfer between the two almost one-to-one.

> **WHY NOT BOTH?**
> Many developers keep both installed and reach for whichever model handles a given task better that week. Because the habits and even the config concepts overlap, the cost of switching is low.

PART FOUR

Best practices & prompting

Habits that make either tool dramatically more useful — and safer.

4.1 Prompt like a manager, not a typist

- **Describe the outcome and the constraints.** "Add pagination to the orders API, 20 per page, keep the existing response shape, add tests." Not "add pagination."
- **Give symptoms for bugs.** What you did, what happened, what you expected. Paste the real error.
- **Number the steps** for multi-part work — it keeps the agent organised and lets you check each stage.
- **Let it explore first.** Ask it to investigate and propose a plan (use `plan` / **Read Only** mode) before it edits anything large.
- **Ask for tests.** "Add a test that fails before your fix and passes after" turns vague confidence into proof.

4.2 Manage context

- Start a fresh conversation (`/clear` in Claude Code) when you switch tasks — a clean context is sharper and cheaper.
- Keep instruction files (`CLAUDE.md` / `AGENTS.md`) short and high-signal. Link out to long docs rather than pasting them.
- Record corrections as new rules in the instruction file so the same mistake doesn't recur.

4.3 Stay safe

- **Always be in Git.** Commit before big tasks; review the diff before you accept it.
- **Use branches and worktrees** for risky or parallel work so `main` stays clean.
- **Default to asking.** Keep the supervised mode until you trust a particular workflow.

- **Never point full-autonomy at production** credentials or systems. Use containers, VMs, or read-only access for unattended runs.
- **Review what it runs.** Read commands before approving, especially anything that deletes, force-pushes, or touches infrastructure.

4.4 Grow into the tooling

You don't need skills, hooks, MCP, or subagents to be productive. Add them when a real pain shows up: reaching for the same multi-step routine (→ a skill), wanting auto-formatting (→ a hook), needing data from Jira or Drive (→ MCP), or facing a task big enough to split (→ subagents). Let need pull you forward rather than configuring everything up front.

PART FIVE

Cheat sheets & glossary

Tear-out references for daily use, plus the vocabulary in one place.

5.1 Claude Code quick reference

Install (mac/Linux): `curl -fsSL https://claude.ai/install.sh | bash`

Start: `cd project && claude`

One-off: `claude -p "explain this file"`

Continue / resume: `claude -c` · `claude -r`

Update: `claude update`

Switch model: `/model` or `--model opus`

Permission cycle: `Shift`+`Tab`

Plan (read-only): `--permission-mode plan`

Clear context: `/clear`

Help / commands: `/help` · type `/`

MCP tools: `claude mcp`

Memory file: `CLAUDE.md` in repo root

Worktree: `claude -w feature-x`

Docs: `code.claude.com/docs`

5.2 OpenAI Codex quick reference

Install (CLI): `npm i -g @openai/codex`

Start: `cd project && codex`

Headless: `codex exec "draft release notes"`

Upgrade: `npm i -g @openai/codex@latest`

Switch model: `/model` or `--model gpt-5.5`

Full auto (careful): `codex --full-auto "…"`

Sandbox: `--sandbox workspace-write`

Local review: `/review`

Memory file: `AGENTS.md` in repo root

Config: `~/.codex/config.toml`

Windows: use WSL

Docs: `developers.openai.com/codex`

5.3 Glossary

TERM	MEANING
Agentic	The tool plans and takes multi-step action toward a goal, not just single suggestions.
Agentic loop	Gather context → plan → act → observe → iterate, repeating until done.
CLI / TUI	The command-line tool you run in a terminal (TUI = its full-screen text interface).
Headless / non-interactive	Running a single task without the interactive UI — `claude -p` / `codex exec`.
CLAUDE.md / AGENTS.md	A Markdown file of project instructions the agent reads each session.
Permission / approval mode	How often the agent pauses to ask before editing files or running commands.
Sandbox	A restricted environment limiting what the agent can read, write, or reach.
MCP	Model Context Protocol — an open standard for connecting agents to external tools and data.
Skill	A packaged, reusable workflow you can invoke (often as a slash command).
Hook	Your own command run automatically around the agent's actions (e.g. auto-format).
Subagent	A helper agent handling part of a task in parallel under a coordinator.
Worktree	An isolated Git working copy so risky/parallel work doesn't disturb your main branch.
Context window	How much text (code + conversation) the model can consider at once.

5.4 Where to go next

- **Claude Code docs:** code.claude.com/docs — quickstart, common workflows, best practices.
- **OpenAI Codex docs:** developers.openai.com/codex — quickstart, CLI, config, security.
- **Practice project:** pick a small repo you know, make a branch, and try one workflow from Part 1.6 or 2.6 end to end. Doing beats reading.

This guide summarises publicly documented behaviour as of May 2026. Both tools update frequently — when a detail here differs from what you see on screen, trust the tool and its official docs, which are the live source of truth. Model names and version numbers in particular change often.